```
    p->trapframe->x0 = -1;
  }
}
```

syscalls[] 是一张以系统调用号为下标的函数指针表。处理函数先检查调用号是否越界，越界就返回 −1，而不是像教学内核那样直接停机——一个用户程序传错调用号，不应该让整个系统死掉。

参数不需要搬运：用户态执行 svc 时它们就在 x0 ~ x5 中，现在原样躺在 trapframe 里，各个 sys_* 函数用 argint()/argaddr()/argstr() 按编号取出：

```
static uint64 argraw(int n)
{
  struct proc *p = myproc();
  switch (n) {
  case 0: return p->trapframe->x0;
  case 1: return p->trapframe->x1;
  ...
  case 5: return p->trapframe->x5;
  }
  panic("argraw");
  return -1;
}
```

指针参数只取出数值，不在这里检查合法性；真正访问用户内存时由 copyin()/copyout() 查用户页表，非法地址会使系统调用返回 −1，而不会让内核出错。

返回值写回 p->trapframe->x0，而不是直接放进 x0 寄存器。原因是返回用户态时 urestorereg 会从 trapframe 恢复所有寄存器，直接写寄存器的值会被覆盖；只有改写保存的那一份，用户程序在 svc 之后才能在 x0 中看到它。

8.1.5 两个栈指针与返回级别

EL0 和 EL1 各有自己的栈指针。从用户态陷入时，硬件立刻把 sp 换成 SP_EL1（内核栈），用户栈指针原样留在 SP_EL0 中。即使内核从不使用 SP_EL0，也必须把它连同其他寄存器一起保存和恢复：陷入期间可能发生进程切换，下一个返回用户态的进程需要的是它自己的用户栈指针。所以 usavereg 把 SP_EL0 存进 trapframe 的 sp 字段，urestorereg 把它写回。

内核态的 savereg 则不需要恢复 sp：内核栈是同一个，即使中间发生了进程切换，swtch() 也已经把 sp 换成了正确的值。

eret 回到哪一级，由 SPSR_EL1 的模式字段决定，不需要另外指定，所以总是回到异常发生时所在的那一级。利用这一点，内核可以“伪造”一次从未发生过的异常来进入用户态：userinit() 在第一个进程的 trapframe 中写入 elr = 0、spsr = 0（模式 EL0t）、sp = PGSIZE，usertrapret() 一执行 eret，CPU 就第一次来到 EL0（第 5 章）。启动时从 EL2 降到 EL1 用的也是同一个技巧（第 3 章）。

8.2 从 init 到 shell

第一个用户进程 initcode 执行 exec("/init")（第 5 章）。user/init.c 做的事情相当于一个嵌入式 Linux 的启动脚本：

1. 创建 /dev，用 mknod 建立 console、tty、ttyS0、input/event0、video0 设备节点；以 /dev/ttyS0 打开 fd 0，复制为 fd 1、2；
2. 创建 /proc、/boot、/mnt/ext2、/etc、/root、/bin 等目录，为根目录下的程序在 /bin 中建立硬链接；